haven’t included it in this chapter because some of the scenarios required provider states to be set up, which we’ll cover in chapter 6.

4.4.1 Committing all changes to GitHub

When you finish this chapter, don’t forget to commit all the changes you’ve introduced locally to your forked GitHub project. To commit all your changes and push them up to your forked project, enter the following commands into your terminal:

```
git add .
git commit -m "added consumer contract tests"
git push origin setup-contract-tests-consumer
```

Summary

- The focus of contract testing is to verify what the consumer needs and check if they get it, based on the contract agreement.
- When writing consumer contract tests, you need to use a mock instead of a stub. A mock validates the behavior of your test, whereas a stub does not.
- Use loose matching, a functionality that allows for more flexible matching of data, when needed, to avoid flaky and brittle contract tests.
- Contract tests, similar to unit tests, should be isolated and not dependent on other tests.
- To follow the examples in the book, Node.js needs to be installed on your machine.
- Movies API is an example API that we’ve used to demonstrate how to write contract tests.
- Pact provides a set of functionalities for setting up the mock provider that the consumer interacts with, defining the expected interactions, and matching rules for a robust test.
- Pact also supports other programming languages such as Java, C#, Ruby, and Python.
- PactFlow, a commercial platform for sharing and managing contracts, is a great way to get started with using Pact Broker.